Learning Complex Physical Systems

Francesco Cenni Francesco Giordani Giulio Mazzaroppi Giacomo Pezzolati

University of Bologna

{*francesco.cenni3, francesco.giordani5, giulio.mazzaroppi, giacomo.pezzolati*}*@studio.unibo.it*

AI in Industry
Academic Year 2025/2026

Overview

1. Introduction & Motivation
2. Modeling and data generation
3. Data Inspection
4. Black Box Approaches
5. Parameter Identification
6. Naive Approaches
7. Neural Networks with Physical Structure
8. Neuro Probabilistic Model
9. Conclusions and Future Works
10. Appendix

Candidates



Francesco Cenni



Francesco Giordani



Giulio Mazzaroppi



Giacomo Pezzolati

Idea of the project

- **Problem context**: modeling non-linear dynamical systems
- **Why a double pendulum**: well-known, relatively simple dynamics, chaotic behavior
- **Industrial relevance**:
  - Benchmark for ML & DL for physical systems
  - Applicable to more complex fields (e.g. **robotics**, **aerospace**)
- **Applications**: control of multi-articulated systems, energy optimization, short-term prediction



Figure: Robotic manipulator motion control



Figure: Impedance control

Double Pendulum Setup

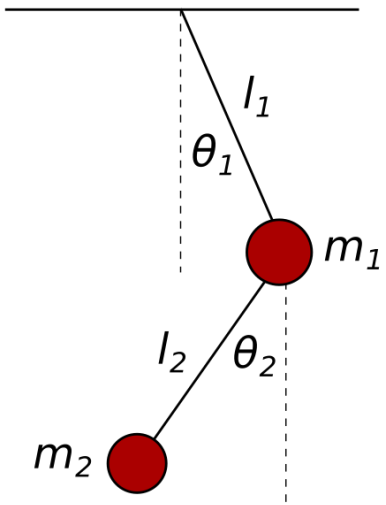


Figure: Double Pendulum Model.

State Variables

$$\mathbf{q} = \begin{bmatrix} q_1 \\ q_2 \\ \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} = \begin{bmatrix} \theta_1 \\ \theta_2 \\ \dot{\theta}_1 \\ \dot{\theta}_2 \end{bmatrix} \in \mathbb{R}^4 \quad (1)$$

Where:

- $\begin{bmatrix} q_1 \\ q_2 \end{bmatrix} \in \mathbb{R}^2$ are the angular positions.
- $\begin{bmatrix} \dot{q}_1 \\ \dot{q}_2 \end{bmatrix} \in \mathbb{R}^2$ are the angular velocities.

Double Pendulum Dynamics

From the Euler-Lagrange equations:

$$\frac{d}{dt}\left(\frac{\partial \mathcal{L}}{\partial \dot{q}_i}\right) - \frac{\partial \mathcal{L}}{\partial q_i} + \frac{\partial \mathcal{R}}{\partial \dot{q}_i} = \tau_i, \quad i = 1, 2 \tag{2}$$

where $\mathcal{L} = T - V$ is the Lagrangian.

- T is the kinetic energy.
- V is the potential energy.
- R is the Rayleigh dissipation function.
- τ_i are the generalized forces (torques) applied at the joints.

The dynamic model of the double pendulum can be expressed in the following matrix form.[1]

Dynamics in Matrix Form

$$M(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + D\dot{\mathbf{q}} + G(\mathbf{q}) = \tau \tag{3}$$

[1] Explicit expression reported in the appendix A

Data Generation

Integration:

- **Forward Euler:**
  - ✓ simple
  - × less accurate
- **Runge-Kutta 4th Order (RK4):**
  - ✓ more accurate
  - × computationally expensive

Simulation Setting

- Duration $T = 10$ s
- Step size $\Delta t = 0.01$ s
- Initial state $q_0 \sim \mathcal{U}[-\pi, \pi], \dot{q}_0 \sim \mathcal{U}[-2, 2]$
- Physical parameters m_1, m_2, l_1, l_2, g(gravity), b_1, b_2(friction)

Generated datasets

- **No friction**: $b_1 = b_2 = 0$
- **Low friction**: $b_1 = b_2 = 0.3$
- **High friction**: $b_1 = b_2 \approx 3.0$ *Excluded for rapid energy dissipation*

Motivation for Learning Dynamics

Why do we need data-driven approaches?

- **Complexity**: real physical systems are chaotic and highly non-linear
- **Cost**: accurate parameter estimation via sensors is expensive and prone to noise
- **Adaptability**: analytical models fail when system properties change over time (e.g., wear and tear)

Data Inspection: Cahotic Behaviour

As an add-on to the data inspection phase, a single pendulum simulation has been performed to compare the distribution of states with the double pendulum one.



Single Pendulum Histogram



Double Pendulum Histogram (No Friction)

States Statistics (1)

During the data inspection phase, we produced visualizations to understand the dataset characteristics, specifically analyzing **Variance** and **Energy** profiles.

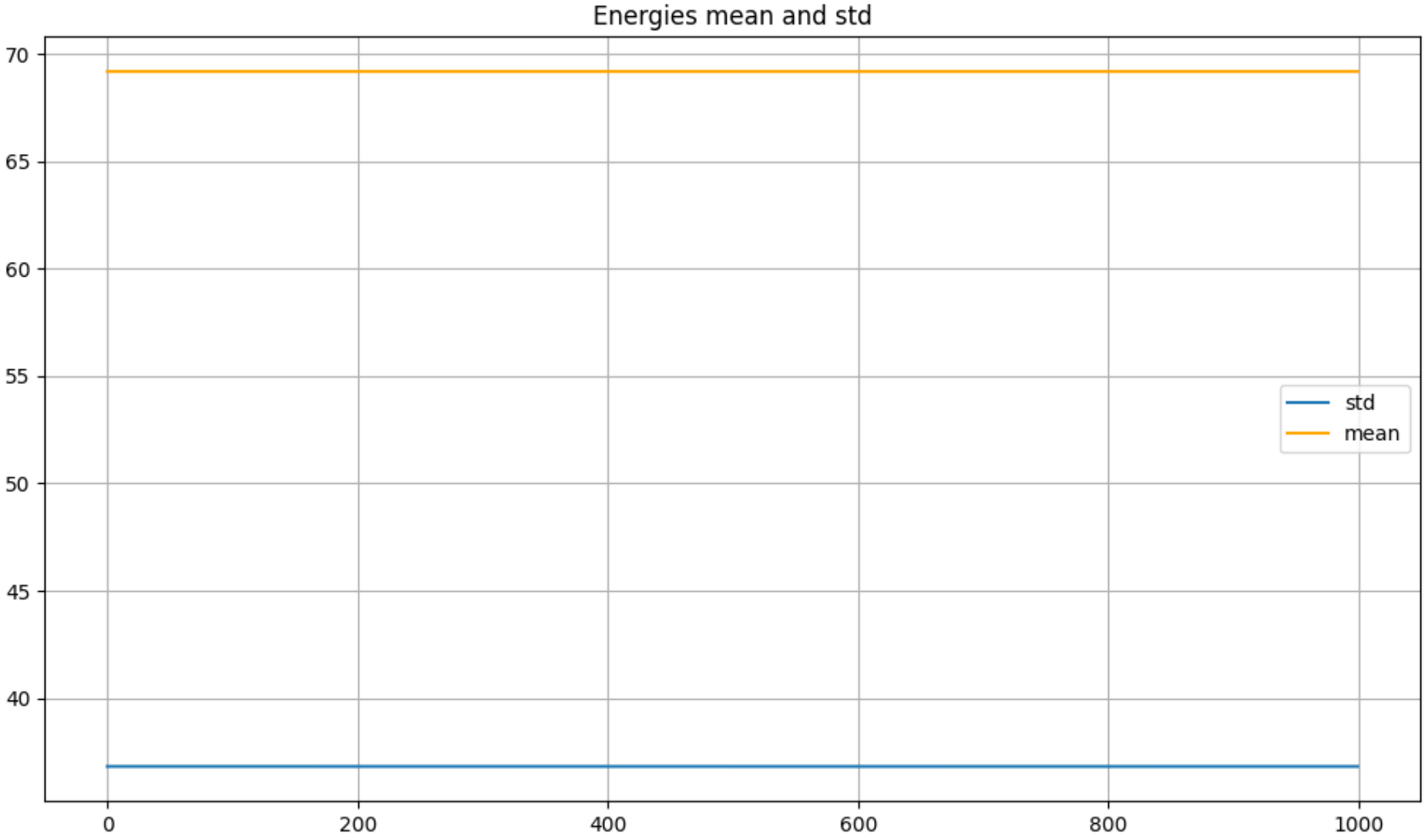


States Statistics (2)



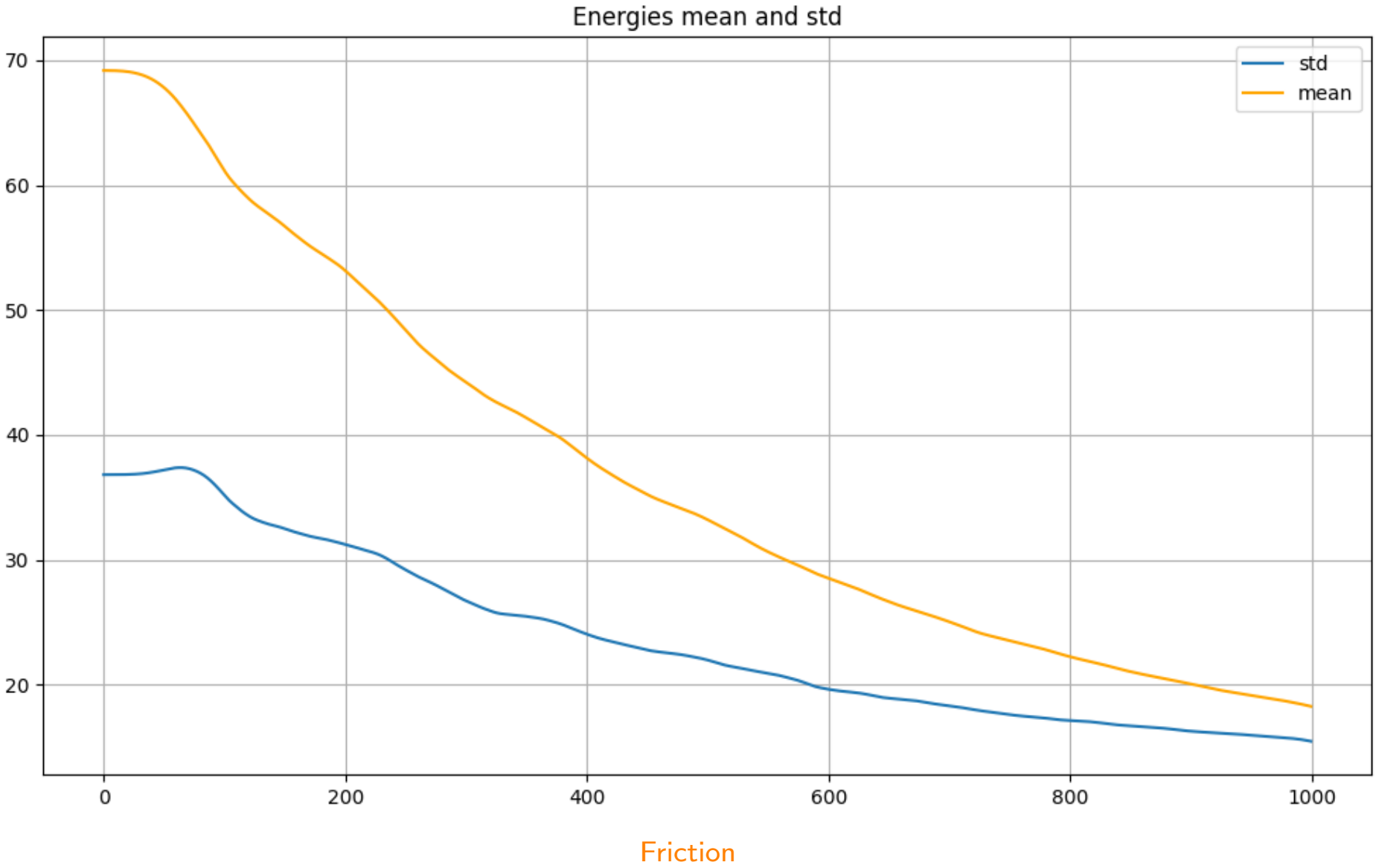
Energy Statistics (1)

Energy statistics were also computed to assess the conservation properties of the simulated systems under different friction conditions.



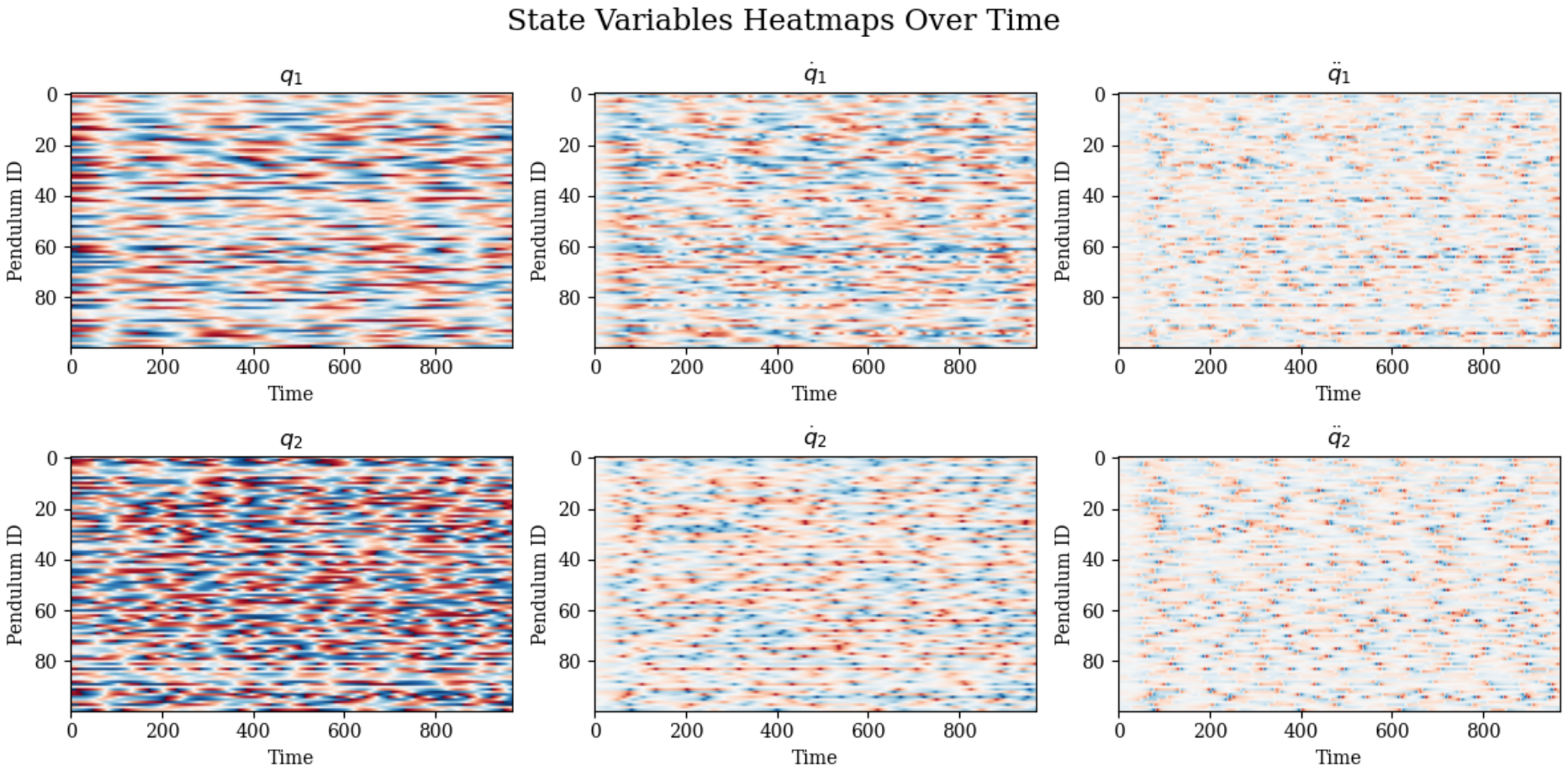
No Friction

Energy Statistics (2)



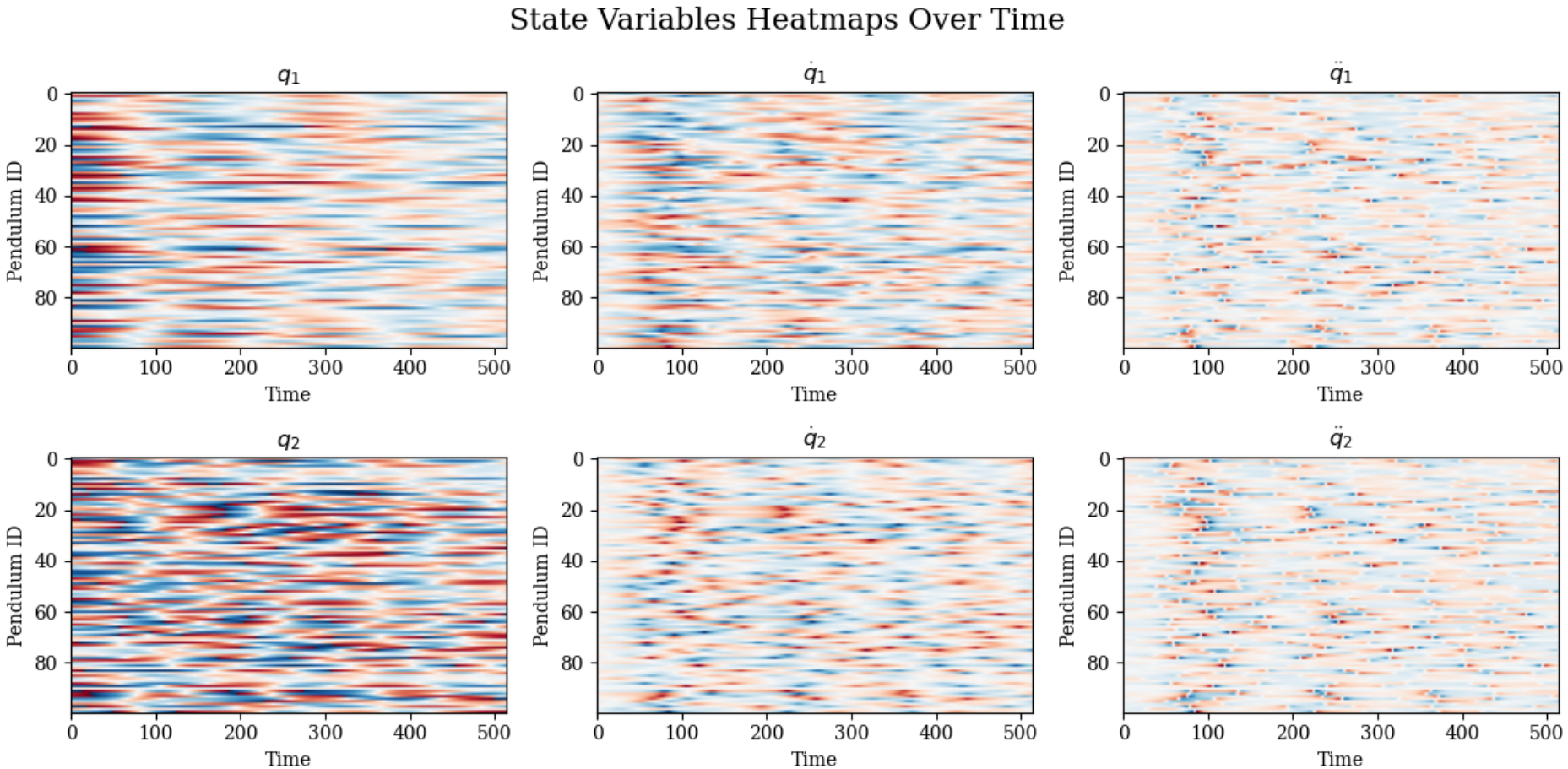
Heatmap (1)

Heatmaps of the states distribution were created to visualize the density of states in the phase space for both friction scenarios.



No Friction

Heatmap (2)



Friction

Pre-processing

Based on the data inspection results, a preprocessing pipeline was designed to enhance data quality and model training effectiveness.

- **Filtering out evolution with increasing energy**: Removing unstable or numerically faulty simulations.
- **White Noise**: Added noise to simulate reality and add robustness to the system.
- **KL Divergence + Cut**:
  - Calculation of the **Kullback-Leibler Divergence** between actual distribution and uniform distribution. Cut of the dataset at the optimum threshold K^* that minimizes the divergence.
  - Identification and removal of the part of the dataset in which the dynamics is less representative.

Data Preprocessing: Dynamic Time Cut Inspection

Following the KL Divergence analysis, the dataset was trimmed to exclude less representative dynamics, enhancing model training quality.

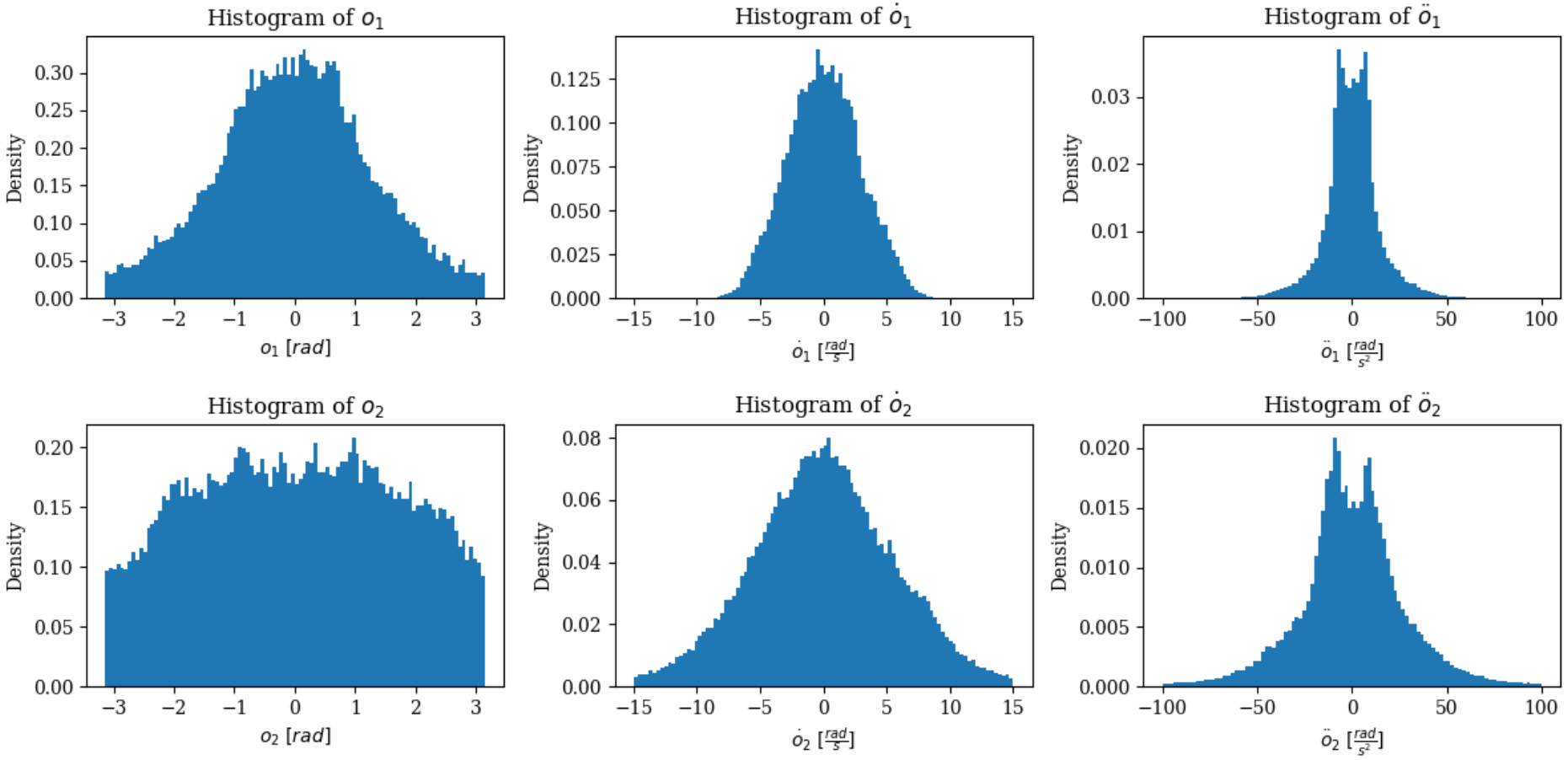


No Friction



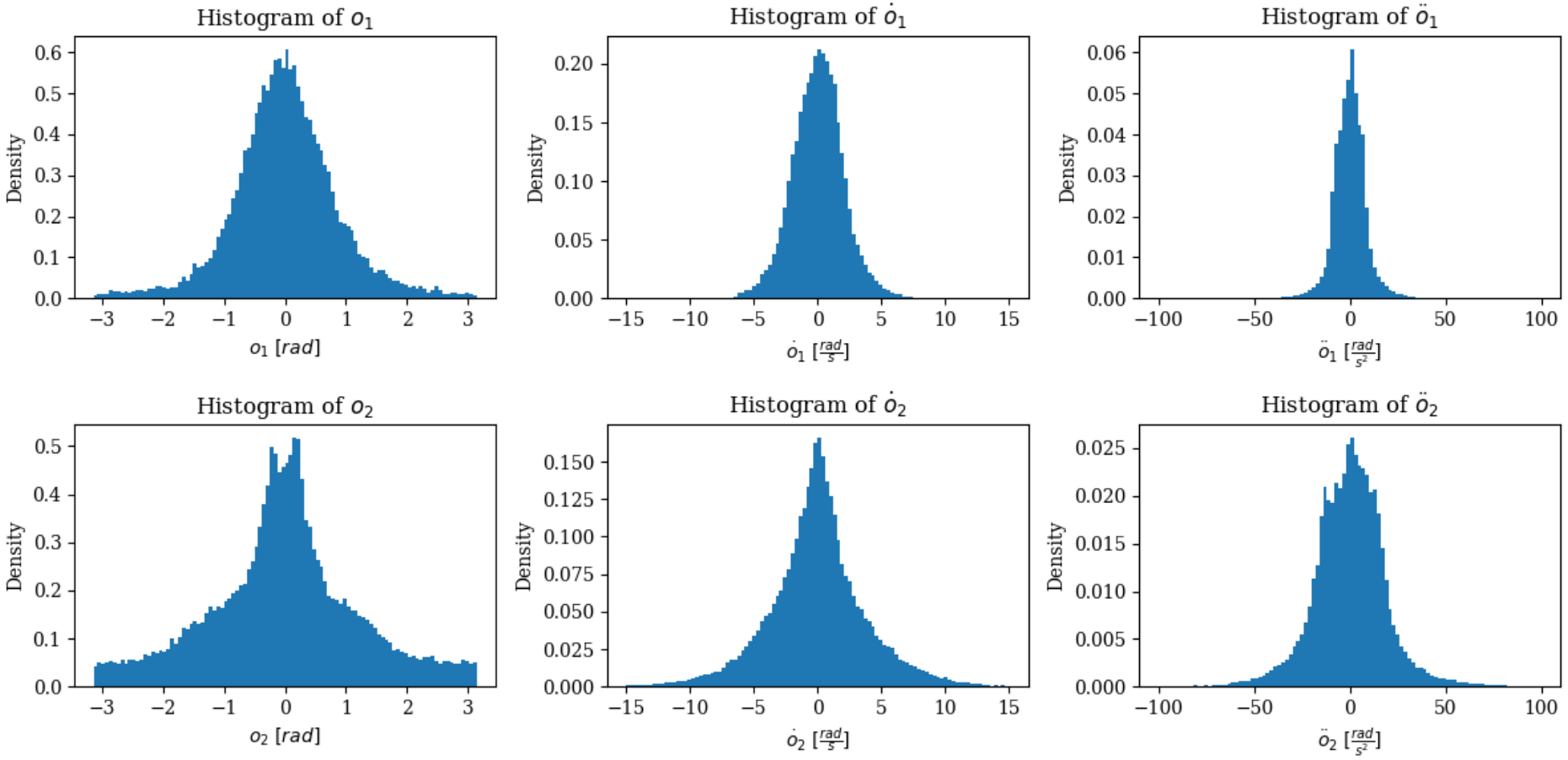
Friction

Pre-Processing: States Distribution (1)



No Friction

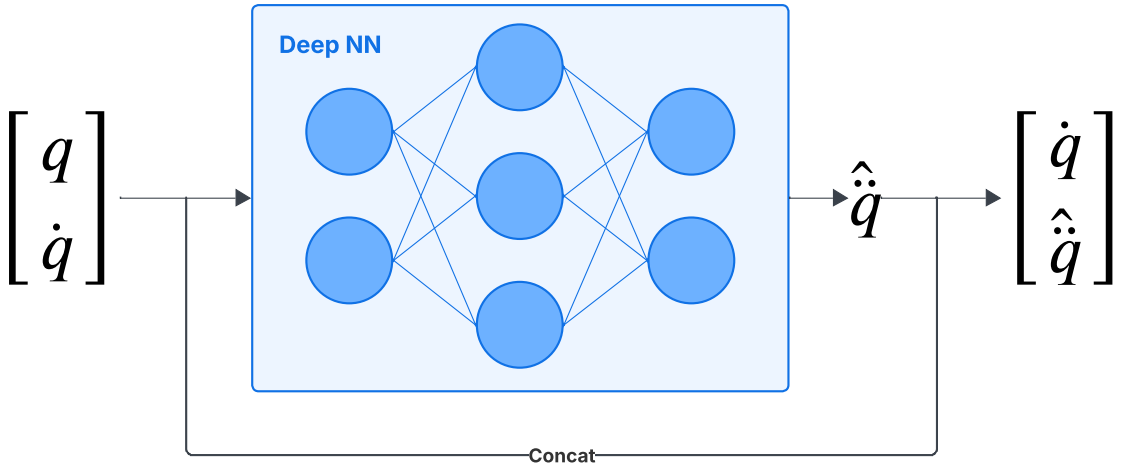
Pre-Processing: States Distribution (2)



Friction

Black Box Approaches - Architecture

Standard Neural Network for setting a **baseline**. The objective is to learn the **dynamics** of the double pendulum.



The model takes as inputs the **states** and outputs the **derivatives**.

Training methods

- **Derivatives**: loss is computed directly on the derivatives.
- **Evolution**: the derivatives are fed to a **solver** that outputs the next state. A sequential stream of predicted states is used to compute the loss.

Evolution Training

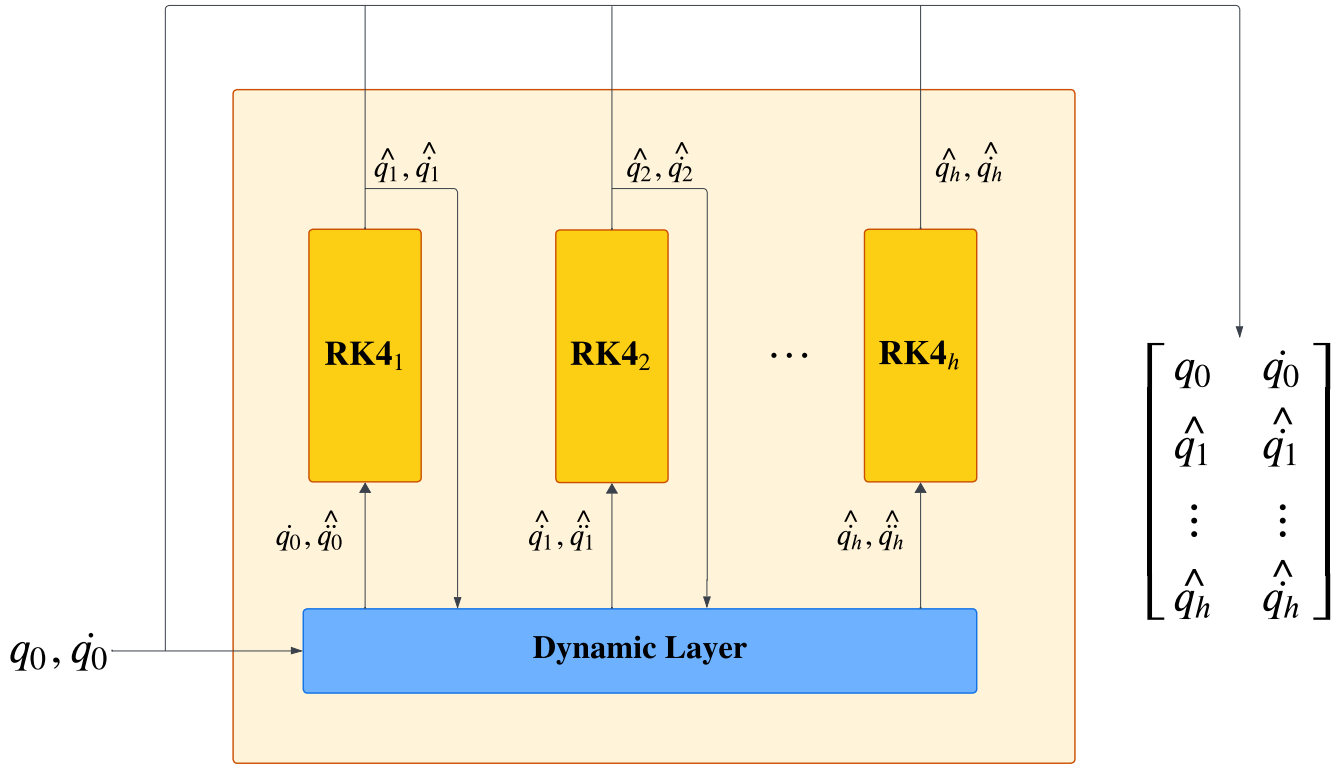


Figure: Evolution training architecture using RK4

Training Related Aspects

Curricular Learning: progressively increasing the integration step h during training

Black Box Approaches - Training recipe (1)

Trigonometric Transformation

Since $\theta \in [-\pi, \pi]$, it's impossible to use the angles directly as inputs without introducing **discontinuities**. The solution is to transform the input space.

$$[\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2]^\top \rightarrow [\cos\theta_1, \sin\theta_1, \cos\theta_2, \sin\theta_2, \dot{\theta}_1, \dot{\theta}_2]^\top$$

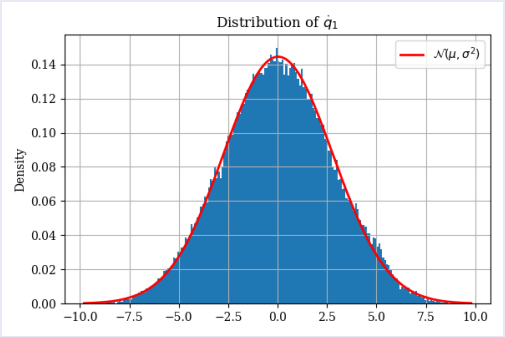
Normalization

Angles

$$-1 \leq \cos\theta \leq 1$$

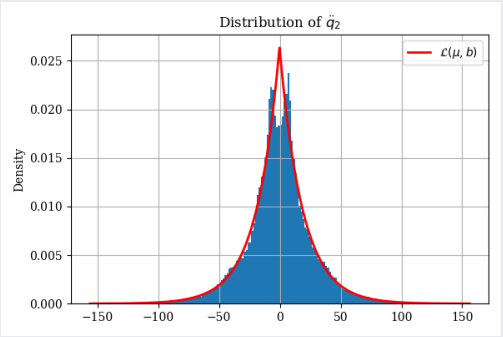
$$-1 \leq \sin\theta \leq 1$$

Velocities



$$\hat{\dot{\theta}} = \frac{\dot{\theta} - \mu}{\sigma}$$

Accelerations



$$\hat{\ddot{\theta}} = \frac{\ddot{\theta} - \tilde{\mu}}{\sqrt{2b}}$$

a

a with b $E[|a - \tilde{a}|]$

Black Box Approaches - Training recipe (2)

Sample weights

Computed using the 4D KDE of the input distribution. In particular:

$$\sigma(\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2) = \frac{h(\theta_1, \theta_2, \dot{\theta}_1, \dot{\theta}_2)}{h_{max}}$$

It's important to use the multivariate distribution to capture any **coupling term** in the dynamics



Curriculum learning

Used specifically for **evolution** learning.
The **window size** of training is increased periodically during training, usually by doubling.
Starting with a window size large from the start is infeasible due to high gradients.

Black Box Approaches - Results

Derivatives Training



Figure: derivatives testing

Evolution Training



Figure: evolution testing



Figure: evolution testing

Black Box Approaches - Considerations

- **Drift Energy** over long horizons



Figure: energy drift

- Low **expressivity** of the model
- Very low **sensitivity** to broken physics

Further developments

- Introduce **Energy conservation**
- Improve **expressivity** and **parameter identification**.

White Box Approach

Physical Parameters Estimation

Assumptions: full Inowledge of the dynamics
Goal: identify physical parameters $m_1, m_2, l_1, l_2, b_1, b_2$.



Physical Parameters Estimation - Architecture

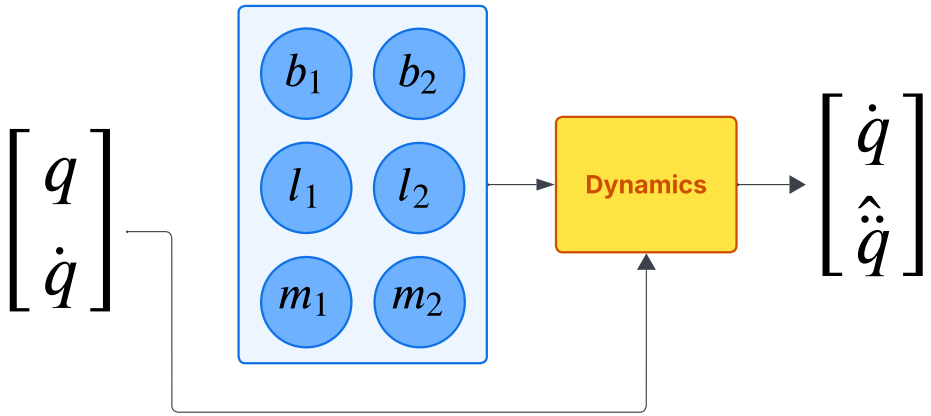


Figure: Dynamic Layer for ODE

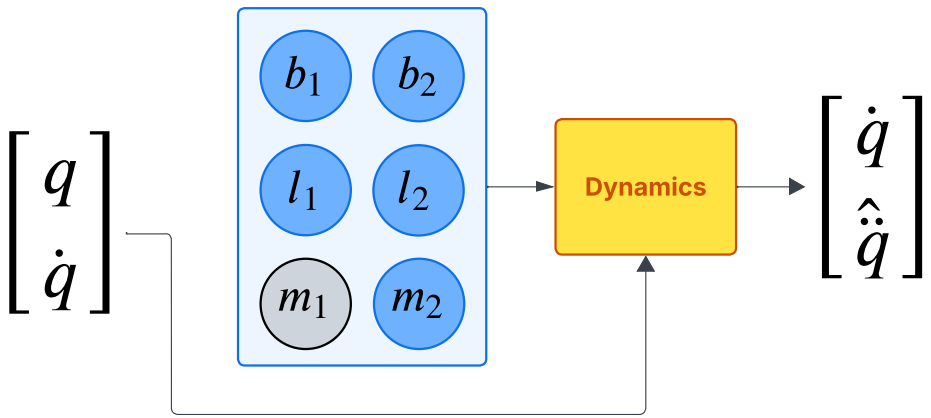


Figure: Dynamic Layer for ODE with fixed parameters

Physical Parameters Estimation - Results

Observations:

- **Gauge Invariance Property**
- Convergence
- **Correct Evolution** $\rightarrow$ deviations over long time.
- **Energy evolution**: matching (*by-construction*).
- low **Max Parameter Discrepancy**

Evolution Testing



Figure: evolution testing

Hamiltonian Neural Networks (HNN)

Choosing the Hamiltonian would be the **best option** but a set of **canonical variables** is needed.

Canonical Variables

$$\mathbf{z} = \begin{bmatrix} \mathbf{q} \\ \mathbf{p} \end{bmatrix} \tag{4}$$

- **q**: generalized displacements
- **p**: generalized momenta

Given the **Hamiltonian** as $\mathcal{H}$, it's possible to find the dynamics with:

$$\frac{dq}{dt} = \frac{d\mathcal{H}}{dp}, \frac{dp}{dt} = -\frac{d\mathcal{H}}{dq}$$

It's possible to use an MLP neural network that learns the Hamiltonian $\mathcal{H}_\theta$ and a loss that enforces the dynamics:

$$\mathcal{L}_{HNN} = \left\| \frac{d\mathcal{H}_\theta}{dp} - \frac{\partial q}{\partial t} \right\|_2 + \left\| \frac{d\mathcal{H}_\theta}{dq} + \frac{\partial p}{\partial t} \right\|_2$$

Two problems of momenta:

- they depend on the parameters of the system
- more importantly they depend on the coupled dynamics not known

Entries Prediction

Remembering equation (4), is it necessary to know matrices structure thoroughly?

Matrices Structure

Goal $\rightarrow$ **Graybox** approach + insights on system properties.
GT (Ground Truth) $\rightarrow$ **Newton-Euler** algorithms.
Application: **Quick inference** for control purposes.

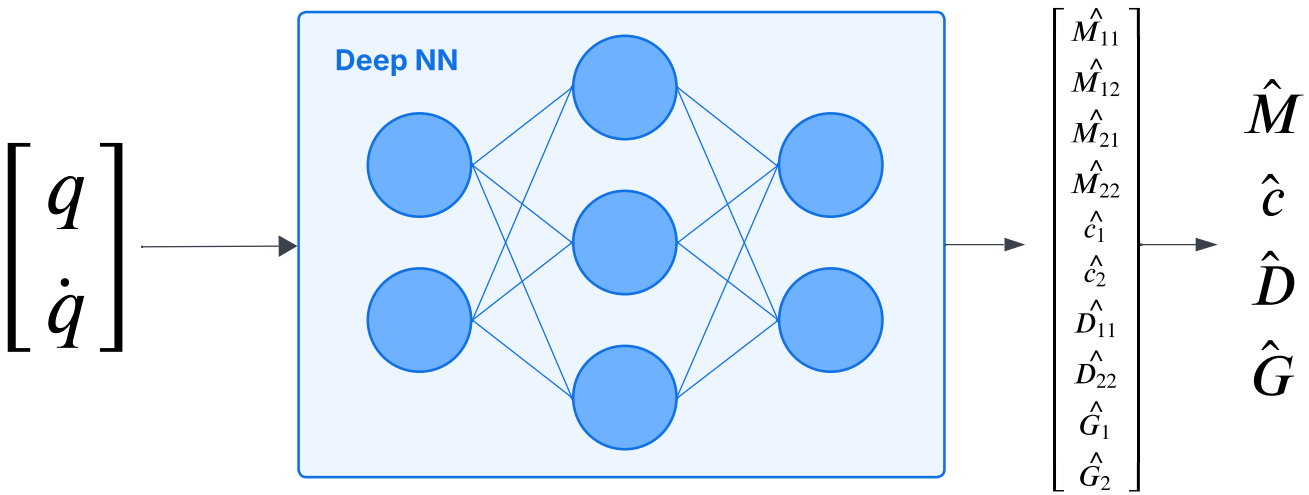


Figure: Entries prediction architecture

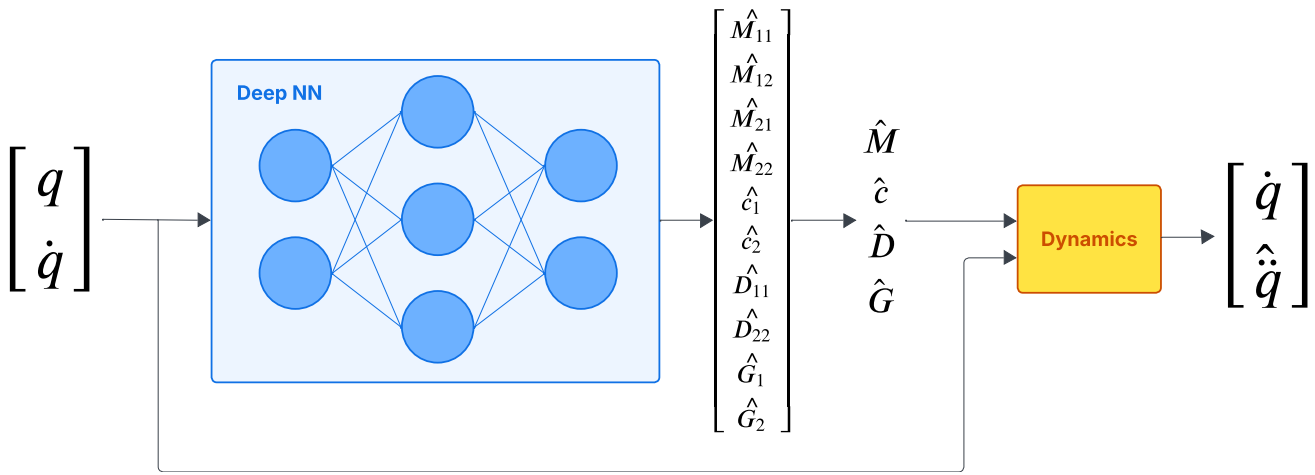
Entries Prediction - Considerations

- **Advantages**:Quick training, very good results.
- **Disadvantages**: Requires GT values for matrices entries.
- We would like to obtain the same results **without** knowing the entries.



UDE – Uninformed

Premise: **Universal Differential Equations**.
Model Layers: Can we exploit past Architecture?.



UDE – Uninformed - Considerations

Matrix Inversion

The equations of motion can be rewritten as:

$$\ddot{\mathbf{q}} = \mathbf{M}(\mathbf{q})^{-1}[\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}})\dot{\mathbf{q}} + \mathbf{D}\dot{\mathbf{q}} + \mathbf{G}(\mathbf{q})]$$

- Much time to converge: RK4 integration combined with deep backpropagation.
- Not constrained : no physical knowledge, not even good sense.
- **Initial Results**: Often **poor results** due to instability.
- **Ill-Posing**: Many **degrees of indeterminacy**.

Lagrangian Neural Networks (LNN) - Architecture

Euler-Lagrange equation can be **discretized** and expanded with chain rule

$$\frac{d}{dt}\nabla_{\dot{q}}\mathcal{L} - \nabla_q\mathcal{L} = 0 \rightarrow \left(\nabla_{\dot{q}}\nabla_{\dot{q}}^{\top}\mathcal{L}\right)\ddot{q} + \left(\nabla_q\nabla_{\dot{q}}^{\top}\mathcal{L}\right)\dot{q} = \nabla_q\mathcal{L}$$

Since the hessian $\nabla_{\dot{q}}\nabla_{\dot{q}}^{\top}\mathcal{L}$, which is the **Inertia matrix**, is positive definite:

$$\ddot{q} = \left(\nabla_{\dot{q}}\nabla_{\dot{q}}^{\top}\mathcal{L}\right)^{-1}\left[\nabla_q\mathcal{L} - \left(\nabla_q\nabla_{\dot{q}}^{\top}\mathcal{L}\right)\dot{q}\right]$$

$\mathcal{L}$ is the output of a black box neural network, while the Lagrangian Equation is computed through internal chains of gradients in the network



Lagrangian Neural Networks (LNN) - Results

Evolution testing



Energy drift LNN vs baseline



Lagrangian Neural Networks (LNN) - Considerations

- Almost perfect **Energy** conservation
- positive definiteness of **Hessian** is not guaranteed being $\mathcal{L}$ black box, causing **instability**
- **Batch Size**: Very small $\rightarrow$ need for **noisy gradients** to escape shallow loss regions (as suggested in literature).
- **Speed Issues**: Hessian computation is expensive.
- **Anomaly**: Difficulty learning **non-conservative systems** or systems with **broken physics**.



Figure: loss descent - correct physics

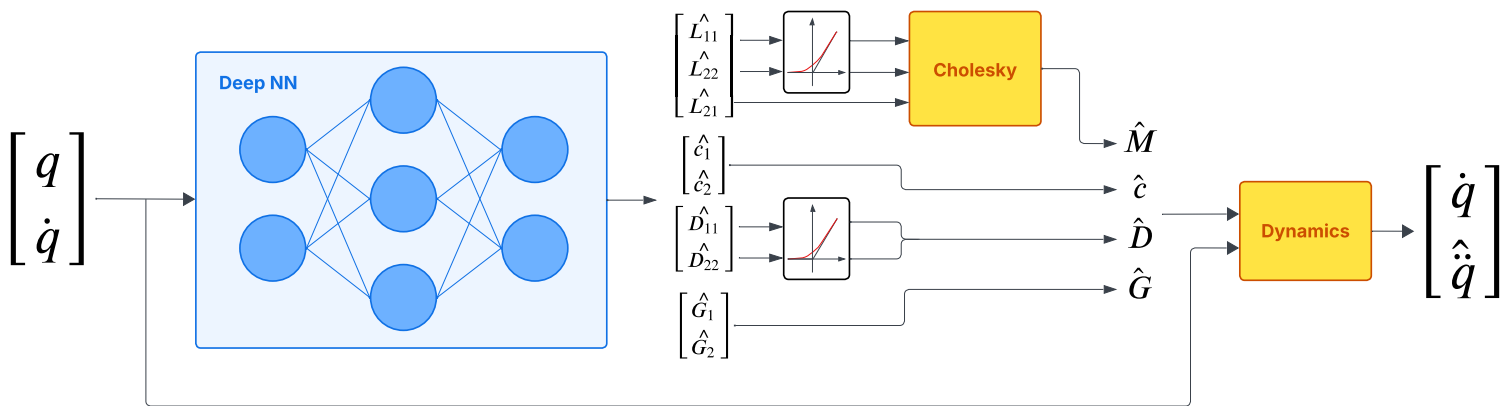


Figure: loss descent - broken physics

Informed UDE

Exploiting All Physical Knowledge:

- Mass matrix **M**: **Positive definite** and **symmetric**.
- Friction: **Positive definite**, **diagonal/constant** matrix.
- Last term in $\mathbf{M}(\mathbf{q})$ matrix is constant as well as $\mathbf{d_{11}}$ and $\mathbf{d_{22}}$ in **D** which is diagonal.
- **Implementation**: Use of **Cholesky decomposition** to guarantee **M** positive definite.



Informed UDE - Results

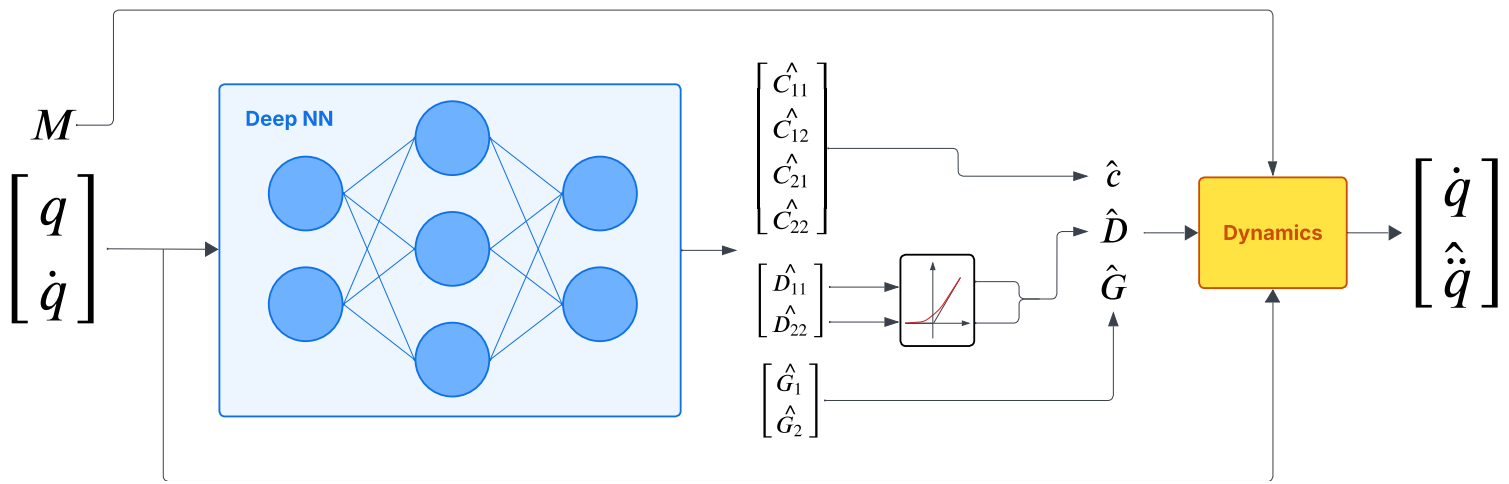
- **Implementation**: Use of **Cholesky decomposition** to guarantee **M** positive definite.
- **Better** takes less to converge, more stable.
- **Troubles** in estimating entries in highly chaotic regions.

Can we plug in even more knowledge?

"More" Informed UDE

Idea

If we somehow could give the matrix M to the system, so as to avoid inversion problems...



Informed UDE - Drawing Conclusions

- **Better** takes less to converge, more stable.
- **Troubles** in estimating entries in highly chaotic regions.

Table: Comparison of the three UDE methods

Metodo	**Time to converge**	**Informations**	**Comments**
Uninformed	High	None	Unstable, Dumb
Informed	Medium	Structural properties	Stable, Complicated Identification
More Informed	Medium	M matrix	Easier Identification, Unrealistic

Structured Lagrangian Neural Network (SLNN) - Architecture (1)

Instead of learning a black box $\mathcal{L}$ it is possible to expolit known physical properties of mechanical systems.

$$\mathcal{L} = T - V = \frac{1}{2}\dot{q}^\top M\dot{q} - V$$

- To learn a positive definite M it is possible to use **Cholesky decomposition**:

  $$M = LL^\top$$

  Where L is a learned **lower triangular** matrix and **softplus** is applied to its diagonal entries.
- The potential energy V is learned via an MPL that has as inputs the state
- Additional: **friction** matrix learned and rendered positive definite via softplus

Structured Lagrangian Neural Network (SLNN) - Architecture (2)



Structured Lagrangian Neural Network (SLNN) - Comparison LNN vs SLNN vs Baseline

Metric	Baseline	LNN	SLNN
Energy conservation	✗	✓	✓
Friction	✓	✗	✓
Broken physics sensitivity	✗	✓	✓
Stable	✓	✗	✓
Not-only mechanical systems	✓	✓	✗

It's possible to add different **modules** to SLNN such as friction to learn different models.

Neuro Probabilistic Model: Motivation

Observation

Standard architectures provide **deterministic** predictions, lacking a **stochastic** explanation of confidence.

- **Goal**: Model the next state **distribution**, assuming a **Gaussian** process.
- **Confidence Prediction**:
  - Prediction is associated with a **confidence contour**.
  - **Diagnosis**: Enables anomaly detection (alarm triggers) when observations fall outside the predicted variance.

Neuro Probabilistic Model: Architecture

To capture aleatoric uncertainty, we modify the Feed-Forward Network (FNN) to output parameters of a probability distribution rather than a single value.

- **Mean** (μ): The expected acceleration.
- **Log-Variance** ($\log \sigma^2$): The estimated uncertainty. Note that we use log for stability.



Neuro Probabilistic Model: Probabilistic Formulation

The model assumes the ground truth acceleration $\ddot{q}$ follows a Gaussian distribution conditioned on the state:

$$p(\ddot{q}|\mathbf{q}) = \mathcal{N}(\ddot{x}|\mu_\theta(\mathbf{q}), \sigma^2_\theta(\mathbf{q}))$$

Training Objective: Negative Log-Likelihood (NLL)

Instead of MSE, we minimize the NLL to learn both the prediction and the uncertainty simultaneously:

$$\mathcal{L}_{NLL} = \frac{1}{2}\log \sigma^2_\theta(\mathbf{q}) + \frac{(\ddot{q}_{gt} - \mu_\theta(\mathbf{q}))^2}{2\sigma^2_\theta(\mathbf{q})} + C$$

Intuition

The model can reduce the penalty of a high error $(\ddot{x} - \mu)^2$ by increasing its uncertainty σ^2, effectively "admitting" it doesn't know the answer.

Neuro Probabilistic Model: Results

Derivatives testing



Evolution testing



Conclusions and Future Works

- **Chaotic Challeng**: It is difficult to predict a chaotic model after several steps.
- **Metrics**: It is better to focus on metrics other than state prediction, as **parameter Identification** and **power spectral density** for instance.

Other points

- **Anomaly Detection** of degraded models or physically wrong ones.
- Developement of a **merged pipeline** that puts together identification and energy conservation.

References


Sam Greydanus, Misko Dzamba, Jason Yosinsky (2019)
Hamiltonian Neural Networks
arXiv (ID: 1906.01563).


Miles Cranmer, Sam Greydanus, Stephan Hoyer, Peter Battaglia, David Spergel, Shirley Ho (2020)
Lagrangian Neural Networks
arXiv (ID: 2003.04630).


Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, David Duvenaud (2018)
Neural Ordinary Differential Equations
arXiv (ID: 1806.07366).

Appendix A - Double Pendulum Dynamics

The dynamic model can be expressed in matrix form as:

$$\boxed{M(\mathbf{q})\ddot{\mathbf{q}} + \mathbf{c}(\mathbf{q}, \dot{\mathbf{q}}) + D\dot{\mathbf{q}} + G(\mathbf{q}) = \tau} \tag{5}$$

Inertia Matrix $M(\mathbf{q}) = M(\mathbf{q})^\top \in \mathbb{R}^4 \succ 0$:

$$\begin{bmatrix} (m_1 + m_2)l_1^2 + m_2 l_2^2 + 2m_2 l_1 l_2 c_2 & m_2 l_2^2 + m2 l_1 l_2 c_2 \\ m_2 l_2^2 + m_2 l_1 l_2 c_2 & m_2 l_2^2 \end{bmatrix}$$

Gravity Vector $G(\mathbf{q})$:

$$\begin{bmatrix} (m_1 + m_2) g l_1 s_1 + m_2 g l_2 s_{12} \\ m_2 g l_2 s_{12} \end{bmatrix}$$

Coriolis $\mathbf{c}(\mathbf{q}, \dot{\mathbf{q}})$:

$$\begin{bmatrix} -m_2 l_1 l_2 s_2 (2\dot{q}_1 + \dot{q}_2)\dot{q}_2 \\ m_2 l_1 l_2 s_2 \dot{q}_1^2 \end{bmatrix}$$

Damping Matrix $D = D^\top \succ 0$:

$$\begin{bmatrix} b_1 & 0 \\ 0 & b_2 \end{bmatrix}$$

Note: $s_1 = \sin(q_1), c_1 = \cos(q_1), s_{12} = \sin(q_1 + q_2), c_{12} = \cos(q_1 + q_2)$.